CHAUDHARY CHARAN SINGH UNIVERSITY (CCSU) • BCA V SEMESTER

BCA-502: Java Programming & Dynamic Webpage Design

Session: Comprehensive PYQ Bank Examination | **Paper Code:** 18022 | **Complete 100% Solved Paper (All 13 Questions)** | **Time:** 3 Hours • **Max Marks:** 75

SECTION – A (VERY SHORT ANSWER QUESTIONS • 3 MARKS EACH • ALL 5 COMPULSORY)

Q1. Why is Java called a Machine Independent language? [3 Marks]

Answer:

Java achieves architecture neutrality through its two-stage compilation and execution process:

> **Compilation & Execution Workflow:**
> `Source Code (.java) → javac Compiler → Intermediate Bytecode (.class) → JVM (Platform-Specific) → Machine Code`

- Java source code is compiled into intermediate binary **Bytecode** rather than platform-specific machine code.
- This bytecode can execute on any operating system (Windows, Linux, macOS) equipped with a suitable **Java Virtual Machine (JVM)**.
- This fulfills the foundational Java paradigm: **"Write Once, Run Anywhere" (WORA)**.

Q2. Explain the functionality and architecture of JVM. [3 Marks]

Answer:

The **Java Virtual Machine (JVM)** is an abstract computing machine that provides the runtime execution environment for compiled Java bytecode. It consists of three primary subsystems:

Subsystem	Components & Core Responsibilities
1. Class Loader Subsystem	Responsible for **Loading** (.class files), **Linking** (Verification, Preparation of static memory, Resolution), and **Initialization** of static blocks and variables.
2. Runtime Data Areas	• **Method Area:** Stores class metadata, bytecode, static variables. • **Heap Area:** Stores all dynamically instantiated objects and arrays. • **JVM Stack:** Stores stack frames for method calls and local variables. • **PC Registers:** Holds memory address of currently executing JVM instruction. • **Native Method Stack:** Holds native C/C++ function call frames.
3. Execution Engine	• **Interpreter:** Reads and executes bytecode instructions line by line. • **JIT (Just-In-Time) Compiler:** Compiles frequently executed "hotspots" directly into native machine code to dramatically improve speed. • **Garbage Collector (GC):** Automatically deallocates unreferenced heap memory.

Q3. Explain the 'super' keyword in Java with examples. [3 Marks]

Answer:

In Java, `super` is a reference variable used by a subclass to refer directly to its immediate parent (superclass) object.

Three Distinct Usages of 'super':

1. **Refer to Hidden Parent Field:** `super.variableName;` resolves variable name collisions when the subclass shadows a parent field.
2. **Invoke Overridden Parent Method:** `super.methodName();` allows a subclass method to explicitly invoke the original parent implementation.
3. **Invoke Parent Constructor:** `super();` or `super(args);` calls the parent class constructor (must strictly be the **first statement** in the subclass constructor).

Q4. Differentiate between Process-based and Thread-based Multitasking. [3 Marks]

Answer:

Comparison Criteria	Process-based Multitasking	Thread-based Multitasking
Definition & Scope	Executing two or more independent heavyweight programs concurrently (e.g., MS Word and Browser).	Executing two or more concurrent paths of execution within the *same* program.
Memory Allocation	Each process is allocated its own distinct, isolated private memory address space by the OS.	All threads belonging to the same process share identical memory address space and Heap memory.
Context-Switch Overhead	High overhead; OS must save registers, memory maps, and page tables.	Very low overhead; fast CPU register swap within shared process memory.
Communication Cost	Inter-Process Communication (IPC) is heavy and requires sockets, pipes, or shared files.	Inter-Thread Communication is fast and efficient using `wait()`, `notify()`, and shared variables.

Q5. Explain Anchor tag (<a>) and Table tag (<table>) in HTML. [3 Marks]

Answer:

- **Anchor Tag (`<a>`):** Creates clickable hyperlinks to navigate to external web pages, files, or internal bookmarks.

```
<a href="https://example.com/syllabus.html" target="_blank" title="View Syllabus">Download Syllabus</a>
```

  Attributes: `href` (target destination URL), `target="_blank"` (opens in new tab), `download` (prompts download).

- **Table Tag (`<table>`):** Organizes complex information into rows and columns.

```
<table border="1" cellpadding="6" cellspacing="0"> <tr><th>Roll No</th><th>Student Name</th> <th>Marks</th></tr> <tr><td>101</td><td>Amit Sharma</td><td>88</td></tr> </table>
```

  Tags: `<tr>` (table row), `<th>` (header cell, bold and centered), `<td>` (data cell), `colspan` / `rowspan` (cell merging).

SECTION – B (SHORT ANSWER QUESTIONS • 7.5 MARKS EACH • ATTEMPT ANY TWO)

Q6. What is an Exception? How do you handle multiple exceptions and use the finally block? [7.5 Marks]

Answer:

An **Exception** is an abnormal event occurring during program execution that disrupts the normal flow of instructions. Java provides robust exception handling using `try`, `catch`, `finally`, `throw`, and `throws`.

Rules for Handling Multiple Exceptions: Specific subclasses of `Exception` must appear *before* generic parent exceptions; otherwise a compilation error ("unreachable code") occurs.

Complete Working Java Code:

```
public class MultiCatchFinallyDemo { public static void main(String[] args) { try { int[] arr = {10, 20, 30}; int divisor = Integer.parseInt(args[0]); // May throw ArrayIndexOutOfBounds or NumberFormat int result = arr[1] / divisor; // May throw ArithmeticException if divisor == 0 System.out.println("Computed Result: " + result); } catch (ArithmeticException e) { System.err.println("Error: Cannot divide by zero! " + e.getMessage()); } catch (ArrayIndexOutOfBoundsException e) { System.err.println("Error: Please provide a command-line argument!"); } catch (NumberFormatException e) { System.err.println("Error: Input must be a valid integer!"); } catch (Exception e) { System.err.println("Generic Fallback Exception caught: " + e); } finally { // The finally block is GUARANTEED to execute regardless of whether an exception occurred or was handled System.out.println("Finally block executed: Cleaned up system resources and closed streams."); } } }
```

Q7. Explain Applet Life Cycle and write a program to display an image. [7.5 Marks]

Answer:

An **Applet** is a special Java program embedded in a web page and executed inside an applet viewer or Java-enabled browser. Its life cycle is managed by four lifecycle methods in `java.applet.Applet` plus the `paint()` method in `java.awt.Component`:

Applet Life Cycle Stages:

`init()` → `start()` → `paint(Graphics g)` ↔ `stop()` → `destroy()`

- `init()`: Called once upon loading; initializes variables, fonts, and images.
- `start()`: Called after `init()` and whenever the browser revisits the page; starts animation/threads.
- `paint(Graphics g)`: Redraws applet UI whenever screen refresh is required.
- `stop()`: Called when user navigates away; suspends background threads.
- `destroy()`: Called prior to unloading; cleans up memory resources.

Applet Code to Load and Display an Image:

```
import java.applet.Applet; import java.awt.Graphics; import java.awt.Image; /* <applet code="ImageDisplayApplet.class" width="400" height="300"></applet> */ public class ImageDisplayApplet extends Applet { private Image img; @Override public void init() { // Load image from the directory hosting the HTML document img = getImage(getDocumentBase(), "logo.png"); } @Override public void paint(Graphics g) { // Draw image at (x=30, y=30) with original dimensions g.drawImage(img, 30, 30, this); g.drawString("CCSU BCA 5th Sem: Java Applet Image Rendering", 30, 260); } }
```

Q8. Explain Threads and two ways to create Threads in Java. [7.5 Marks]

Answer:

A **Thread** is a lightweight sub-process representing the smallest unit of dispatchable execution. In Java, multithreading is achieved through two standard mechanisms:

Approach 1: Extending the `java.lang.Thread` Class

```
class WorkerThread extends Thread { @Override public void run() { for (int i = 1; i <= 3; i++) { System.out.println("WorkerThread executing step " + i + " on: " + Thread.currentThread().getName()); } } } public class ThreadDemo1 { public static void main(String[] args) { WorkerThread t1 = new WorkerThread(); t1.setName("Thread-Alpha"); t1.start(); // Spawns new OS thread and invokes run() } }
```

Approach 2: Implementing the `java.lang.Runnable` Interface (Recommended)

```
class WorkerTask implements Runnable { @Override public void run() { for (int i = 1; i ⩽ 3; i++) { System.out.println("RunnableTask step " + i + " on: " + Thread.currentThread().getName()); } } } public class ThreadDemo2 { public static void main(String[] args) { WorkerTask task = new WorkerTask(); Thread t2 = new Thread(task, "Thread-Beta"); t2.start(); } }
```

Key Advantage of Runnable: Since Java allows single class inheritance, implementing `Runnable` frees the class to inherit from another domain superclass while still enabling multithreading.

SECTION – C (LONG ANSWER QUESTIONS • 15 MARKS EACH • ATTEMPT ANY THREE)

Q9. Illustrate Inheritance. Why is multiple inheritance not possible in Java? How is it implemented via Interfaces? [15 Marks]

Answer:

Inheritance is an Object-Oriented mechanism whereby a subclass automatically acquires fields and methods from a superclass using the `extends` keyword, fostering code reusability and polymorphism.

The Diamond Problem (Why Multiple Class Inheritance is Prohibited):
If Class C inherits from both Class A and Class B, and both A and B define method `void display()` with different implementations: which version does an instance of Class C inherit? This ambiguity is known as the **Deadly Diamond of Death**. To avoid architectural complexity and pointer issues, Java designers disallowed multiple inheritance of classes.

Resolution using Interfaces:
Since interfaces contain only method declarations without state variables, there is zero ambiguity. A single Java class can implement multiple interfaces simultaneously.

```
interface Printable { void print(); // Abstract contract } interface Showable { void show(); } // Class implementing multiple interfaces class DocumentReport implements Printable, Showable { @Override public void print() { System.out.println("Printing document paper copy."); } @Override public void show() { System.out.println("Showing document on digital screen."); } } public class MultipleInheritanceDemo { public static void main(String[] args) { DocumentReport doc = new DocumentReport(); doc.print(); doc.show(); } }
```

Q10. What is JDBC? Explain types of JDBC drivers. Write a program to insert and select records using PreparedStatement. [15 Marks]

Answer:

Java Database Connectivity (JDBC) is a standard Java API (`java.sql` package) providing uniform, vendor-neutral database communication.

Driver Category	Architecture & Mechanism	Pros & Cons
Type-1: JDBC-ODBC Bridge	Translates JDBC calls into ODBC calls; requires native ODBC client installed.	Slow, platform-dependent, deprecated in modern Java.
Type-2: Native-API Driver	Converts JDBC calls directly into native C/C++ vendor client library calls.	Faster than Type-1, but requires native vendor client binaries on each machine.
Type-3: Network-Protocol Driver	Sends JDBC calls to an intermediate application server middleware.	Flexible, multi-database support, but adds server hop.
Type-4: Thin Driver (Pure Java)	Directly converts JDBC calls into database native network socket protocol.	**Highest performance**, 100% pure Java, platform independent, no client installation.

Complete JDBC Program (PreparedStatement Insert & Select):

```
import java.sql.Connection; import java.sql.DriverManager; import java.sql.PreparedStatement;
import java.sql.ResultSet; public class JdbcPreparedStatementDemo { public static void
main(String[] args) { String url = "jdbc:mysql://localhost:3306/college_db"; String user = "root";
String pass = "password"; try (Connection con = DriverManager.getConnection(url, user, pass)) { //
1. Parameterized INSERT using PreparedStatement (Prevents SQL Injection) String insertSql = "INSERT
INTO students (roll_no, name, marks) VALUES (?, ?, ?)"; try (PreparedStatement psInsert =
con.prepareStatement(insertSql)) { psInsert.setInt(1, 105); psInsert.setString(2, "Vikas Verma");
psInsert.setDouble(3, 91.5); int rows = psInsert.executeUpdate(); System.out.println("Rows
successfully inserted: " + rows); } // 2. Querying data using PreparedStatement String selectSql =
"SELECT roll_no, name, marks FROM students WHERE marks ⩾ ?"; try (PreparedStatement psSelect =
con.prepareStatement(selectSql)) { psSelect.setDouble(1, 75.0); ResultSet rs =
psSelect.executeQuery(); System.out.println("--- High Performing Students ---"); while (rs.next())
{ int rno = rs.getInt("roll_no"); String sname = rs.getString("name"); double scr =
rs.getDouble("marks"); System.out.println("Roll: " + rno + " | Name: " + sname + " | Marks: " +
scr); } } } catch (Exception e) { e.printStackTrace(); } } }
```

Q11. Explain Servlets and their Life Cycle. Compare Servlets and JSP. [15 Marks]

Answer:

A **Servlet** is a server-side Java class managed by a Servlet Container (e.g., Apache Tomcat) that receives client HTTP requests, processes business logic, and generates dynamic web responses.

Servlet Life Cycle (javax.servlet.Servlet Interface):

1. **Loading & Instantiation:** Servlet class bytecode is loaded and a single instance is instantiated in container memory.
2. **Initialization (`init(ServletConfig config)`):** Invoked **only once** immediately after instantiation; loads database connections and parameters.
3. **Request Servicing (`service(ServletRequest, ServletResponse)`):** Dispatches each incoming request thread to handler methods (`doGet()` or `doPost()`). Runs concurrently on separate threads.
4. **Destruction (`destroy()`):** Invoked **only once** when container shuts down or reloads; deallocates resources.

Comparison: Servlets vs JavaServer Pages (JSP):

Feature	Servlets	JSP (JavaServer Pages)
Core Paradigm	Pure Java code with embedded HTML written inside `out.println()` statements.	Standard HTML markup with embedded Java scriptlet tags (`<% ... %>`).
MVC Architectural Role	Best suited as **Controller** to manage request dispatching, validation, and business logic.	Best suited as **View** layer to present and format UI output for end users.
Compilation Speed	Pre-compiled into bytecode; executes rapidly from the very first request.	First-request is slower because JSP is internally translated into a Servlet and compiled.

Q12. Explain TCP/IP Socket Programming in Java with a client-server example. [15 Marks]

Answer:

Java Socket programming provides process-to-process communication across networks using the `java.net` package. TCP provides reliable, stream-oriented, full-duplex transmission.

Core Classes:

- `java.net.ServerSocket` : Listens on a dedicated port and waits for incoming client requests via `accept()` .
- `java.net.Socket` : Connects to a remote server IP and port; provides `getInputStream()` and `getOutputStream()` .

Server Application (ServerApp.java):

```
import java.io.*; import java.net.*; public class ServerApp { public static void main(String[]
args) throws IOException { int port = 5000; try (ServerSocket serverSocket = new
ServerSocket(port)) { System.out.println("Server listening on port " + port + "... Waiting for
client..."); try (Socket socket = serverSocket.accept()) { // Blocks until client connects
System.out.println("Client connected: " + socket.getInetAddress()); BufferedReader in = new
BufferedReader(new InputStreamReader(socket.getInputStream())); PrintWriter out = new
PrintWriter(socket.getOutputStream(), true); String clientMessage = in.readLine();
System.out.println("Received from client: " + clientMessage); out.println("Echo from Server: Hello,
" + clientMessage.toUpperCase()); } } } }
```

Client Application (ClientApp.java):

```
import java.io.*; import java.net.*; public class ClientApp { public static void main(String[]
args) throws IOException { String host = "127.0.0.1"; int port = 5000; try (Socket socket = new
Socket(host, port)) { PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
out.println("CCSU BCA 5th Sem Student"); String response = in.readLine();
System.out.println("Server response: " + response); } } }
```

Q13. Discuss Layout Managers in Java AWT/Swing (FlowLayout, BorderLayout, GridLayout, CardLayout, GridBagLayout). [15 Marks]

Answer:

A **Layout Manager** automatically computes the geometric sizing and spatial positioning of GUI components inside a container (such as `JFrame` or `JPanel`), abstracting away OS platform differences.

Layout Manager	Spatial Arrangement Strategy	Default Container & Behavior
FlowLayout	Arranges components sequentially in a horizontal line. When row width is exhausted, wraps automatically to next line.	Default for `JPanel`. Retains preferred component size.
BorderLayout	Divides container into 5 designated regions: `NORTH`, `SOUTH`, `EAST`, `WEST`, and `CENTER`. Components stretch to fill their region.	Default for `JFrame` and `JDialog`. Unused regions collapse into `CENTER`.
GridLayout	Arranges components in a strict two-dimensional matrix of equal-sized rectangular grid cells (rows × columns).	All components forced to identical width and height regardless of preferred size.
CardLayout	Stacks components on top of each other like a deck of playing cards. Only one component (card) is visible at a time.	Ideal for wizard installers, multi-tab dialogues, and step-by-step forms.
GridBagLayout	The most flexible and sophisticated layout manager. Components can vary in size and span multiple rows/columns using `GridBagConstraints`.	Allows precise, enterprise-grade responsive form alignment without nested panels.